

Modeling YouTube Video Engagement and Trending Popularity

MATH 448 — Progress Report 1

Jannagrace Carandang (924173494)
Uzair Hamed Mohammed (920142896)

Fall 2026

Contents

1	Introduction and Objectives	3
1.1	Objectives	3
1.2	Research questions	3
1.3	Scope of this report	3
2	Data Overview	3
2.1	Source and scope	3
2.2	Unit of analysis	4
2.3	Variables	4
3	Preprocessing	5
3.1	Data-quality audit	5
3.2	Cleaning steps	5
3.3	Derived features and transformations	5
4	Exploratory Data Analysis	6
4.1	The response is severely right-skewed	6
4.2	Category is the strongest categorical signal	6
4.3	Engagement ratios differ by category	7
4.4	Comment/rating settings	8
4.5	Time effects are mild	8
4.6	Text and tag features carry weak linear signal	8
4.7	Collinearity and leakage become visible in the correlation matrix	9
4.8	Class balance and snapshot multiplicity	10
5	Data Issues, Limitations, and Leakage Risks	11
6	Initial Modeling Plan	12
6.1	Analysis unit and targets	12
6.2	Candidate methods and why	12
6.3	Validation framework	12
6.4	Evaluation metrics	12
6.5	Planned comparisons	13
7	Summary and Next Steps	13
8	Appendix: Analysis Code	14
8.1	profile_data.py — data audit	14

8.2	<code>clean_eda.py</code> — cleaning, feature engineering, figures	15
8.3	<code>report_numbers.py</code> — summary statistics cited above	18
9	References	19

1 Introduction and Objectives

YouTube is the largest video-sharing platform in the world, and for a large class of creators and marketers the “Trending” tab is the main discovery surface they compete for. The platform’s own ranking logic is hidden, but creators control a documented set of upload-time decisions — which category to publish in, what time and day to release, how long the title is, how many tags to attach, and whether comments and ratings are enabled. This project studies the relationship between those controllable, upload-time metadata factors and the post-release performance of videos that reached the Trending tab.

1.1 Objectives

Following the project proposal, we pursue three complementary goals:

1. **Regression.** Predict continuous engagement outcomes — in particular $\log(1 + \text{views})$ and engagement ratios such as $\text{likes}/\text{views}$ (like rate) and $\text{comment_count}/\text{views}$ (comment rate).
2. **Classification.** Predict whether a video attains “High Engagement,” defined here as a top-quartile final view count.
3. **Inference.** Estimate whether creator-controlled choices (publish time, title length, tag count, comment/rating settings) have statistically significant effects on audience reception, and in which direction.

The first two are prediction tasks; the third is a statistical-inference task, and the two call for different tools (see Section 6).

1.2 Research questions

1. **RQ1 (regression).** How much of the variation in $\log(1 + \text{views})$, like rate, and comment rate can upload-time metadata explain, and which metadata features carry the most signal?
2. **RQ2 (classification).** Can upload-time metadata distinguish top-quartile-view videos from the rest, and how well (ROC-AUC, error rate)?
3. **RQ3 (inference).** Are the associations between creator choices and engagement statistically significant and stable, after controlling for topic (category)?

1.3 Scope of this report

This first progress report establishes the empirical foundation: it imports and audits the data, defines the analysis unit, preprocesses and engineers features, performs exploratory data analysis (EDA), documents data issues and leakage risks, and lays out the initial modeling and validation plan. Model fitting and final results are deferred to Progress Report 2.

2 Data Overview

2.1 Source and scope

The data come from the “**Trending YouTube Video Statistics**” dataset (Kaggle), which provides daily snapshots of the videos appearing on the Trending tab across several countries. We use the **US** file, `USvideos.csv`, together with the label mapping `US_category_id.json`. The file we analyzed has exactly **40,949 rows and 16 columns**, spanning trending dates from **2017-11-14 to 2018-06-14** (205 distinct days). (We flag a citation check in Section 5: the proposal attributes the set to a specific contributor that should be verified against the Kaggle listing.)

2.2 Unit of analysis

Each row is a **video-day snapshot**: a video appears once for every day it stayed on the Trending tab, with cumulative counts as of that day. This structure is central to the whole project. In the raw file there are only **6,351 unique video_id values** across the 40,949 rows: on average each video appears **6.45** times, up to **30** times, and **88.9%** of videos appear on two or more days. We therefore work at two levels:

1. **Snapshot level** — all 40,876 cleaned snapshot rows, used only to study trending duration and growth.
2. **Video level** — one row per video (its **last** trending snapshot, i.e. final cumulative performance), giving **n = 6,348** videos. This is the unit for modeling. Keeping the **last** snapshot makes the outcome “final cumulative views,” and it is the choice that avoids the most leakage (Section 5).

2.3 Variables

The 16 raw variables and their role in the analysis are summarized in Table 1. Outcomes are the cumulative engagement counts; predictors are upload-time metadata. Several predictors are **derived** from raw text/time fields during preprocessing (Section 3).

Variable	Type	Role	Description
video_id	string	identifier	Unique video key; not a predictor.
trending_date	date	sampling	Day the video appeared in Trending; format YY.DD.MM.
title	text	feature src	Video title; source of title-length/word features.
channel_title	categorical	group/feature	Channel name; 2,207 levels.
category_id	categorical	predictor	Topic bucket; 16 levels used.
publish_time	datetime	feature src	Upload timestamp (UTC).
tags	text list	feature src	Pipe-separated keywords; [none] if absent.
views	integer	outcome	Cumulative views at snapshot.
likes	integer	outcome	Cumulative likes.
dislikes	integer	outcome	Cumulative dislikes.
comment_count	integer	outcome	Cumulative comments.
thumbnail_link	URL	drop	Thumbnail URL; ID-like, no signal.
comments_disabled	boolean	predictor	Comments turned off (rare).
ratings_disabled	boolean	predictor	Ratings turned off (rare).
video_error_or_removed	boolean	filter	Video unavailable; used as a filter.
description	text	drop	Description text (570 missing); unused.

Table 1: Raw variables, types, roles, and descriptions. X = predictor, Y = outcome.

Outcomes. views, likes, dislikes, comment_count; plus derived $\log(1 + x)$ transforms and the ratios like_rate, comment_rate, dislike_rate.

Predictors. category, publish_hour, publish_dayofweek, title_length, title_word_count, tag_count, days_to_trend, comments_disabled, ratings_disabled. The engagement counts and thumbnail_link are **excluded** from X for leakage reasons (Section 5).

3 Preprocessing

3.1 Data-quality audit

Before transforming anything we audited the raw file. Findings:

1. **Missing values.** There is almost none. Only description has nulls (570 rows, about 1.4%), plus 8 empty strings. Since description is not used, no imputation is required.
2. **Sentinels.** tags uses the literal string [none] in **1,535** rows (about 3.8%). This must be mapped to “zero tags,” not counted as one tag.
3. **Duplicates.** 48 fully duplicated rows and 2 duplicate (video_id, trending_date) pairs.
4. **Flags.** video_error_or_removed is TRUE for only **23** rows; comments_disabled and ratings_disabled are TRUE for 1.55% and 0.41% of rows.
5. **Logical consistency.** No violations: no row has likes, dislikes, or comment_count exceeding views, and every row with comments_disabled = TRUE has comment_count = 0 (a structural, not a data-entry, relationship).

3.2 Cleaning steps

The cleaning pipeline and the row count after each step are shown in Table 2. Each step is justified:

1. **Drop video_error_or_removed.** These 23 rows describe videos that are no longer available; their engagement counts are unreliable and they cannot be modeled fairly.
2. **Drop exact duplicates.** 48 rows are byte-identical repeats (data-collection artifacts) and would otherwise double-count those videos.
3. **Drop duplicate video-days.** 2 rows repeat the same (video_id, trending_date) pair; these are redundant snapshots of the same day.
4. **Collapse to one row per video.** We keep each video’s **last** trending snapshot (maximum trending_date), which encodes final cumulative performance and removes the within-video dependence that would otherwise leak across folds (Section 5).

Step	Rows removed	Rows remaining
Raw file	—	40,949
Drop video_error_or_removed,	23	40,926
Drop fully-duplicated rows	48	40,878
Drop duplicate (video_id, trending_date)	2	40,876
Collapse to one row per video (last snapshot)	—	6,348 videos

Table 2: Cleaning pipeline with observation counts. The final analysis table has 6,348 videos.

3.3 Derived features and transformations

1. trending_date is parsed with the format YY.DD.MM (an unusual, easy-to-misread layout: 17.14.11 means 2017-11-14); publish_time is ISO-8601 UTC. days_to_trend is the gap between the two.
2. publish_hour and publish_dayofweek are extracted from publish_time.
3. title_length (characters) and title_word_count (whitespace tokens) are derived from title.
4. tag_count counts pipe-separated tags, with [none] mapped to 0.
5. category is the human-readable name from US_category_id.json.

6. **Log transform.** We use $\log(1 + x)$ for views, likes, dislikes, and comments to stabilize the severe right-skew (Figure 1) while handling the zeros in the like/dislike/comment counts (32, 99, and 140 videos respectively).
7. **Ratios.** `like_rate`, `comment_rate`, `dislike_rate` are computed per view.
8. **Label.** `high_views` = 1 if final views are in the top quartile, i.e. at least **1,473,578** views (a balanced 25% / 75% split).

The final video-level table (`analysis/output/videos.csv`) has $n = 6,348$ rows and is the input to EDA and modeling.

4 Exploratory Data Analysis

4.1 The response is severely right-skewed

Figure 1 contrasts raw views with their log transform. Raw views have skewness **14.2**, with the median (516,214) far below the mean (1,962,282) and a maximum of 225 million; the transform brings skewness to about **-0.5** and a roughly bell-shaped distribution. This directly justifies the log transformation for all count outcomes. (The same pattern holds for likes, dislikes, and comments, whose raw skewnesses are 12.5, 44.9, and 24.1.)

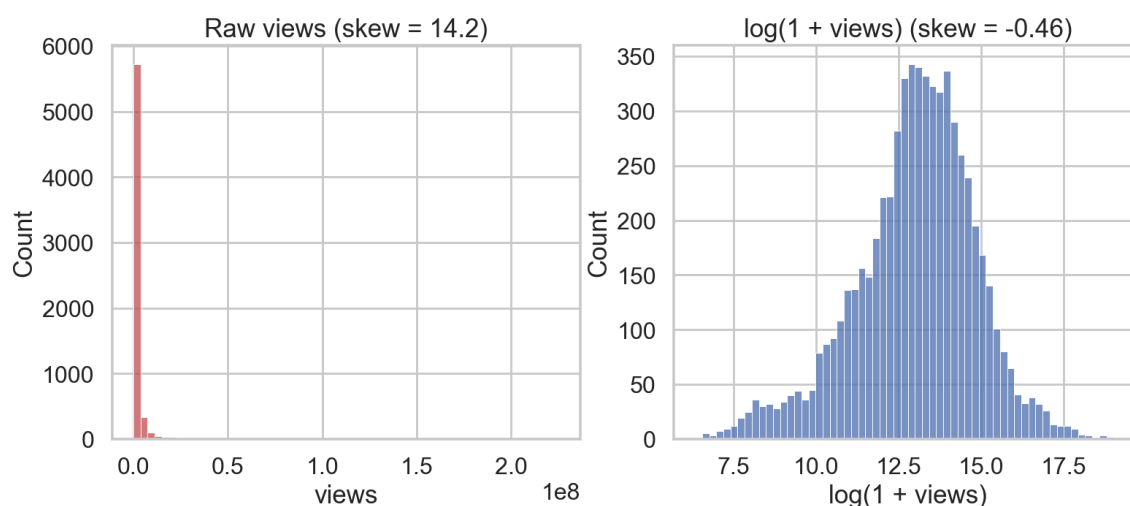


Figure 1: Raw views (left) versus $\log(1 + \text{views})$ (right) at the video level. The log transform removes almost all of the skew.

4.2 Category is the strongest categorical signal

Figure 2 shows $\log(1 + \text{views})$ by category. Gaming and Music have the highest median views, followed by Film & Animation and Comedy; News & Politics and Nonprofits & Activism sit lowest. The spread within categories is large, so category is informative but far from deterministic. Two categories have very few observations (Nonprofits & Activism, $n = 14$; Shows, $n = 4$), which will need care (pooling or regularization).

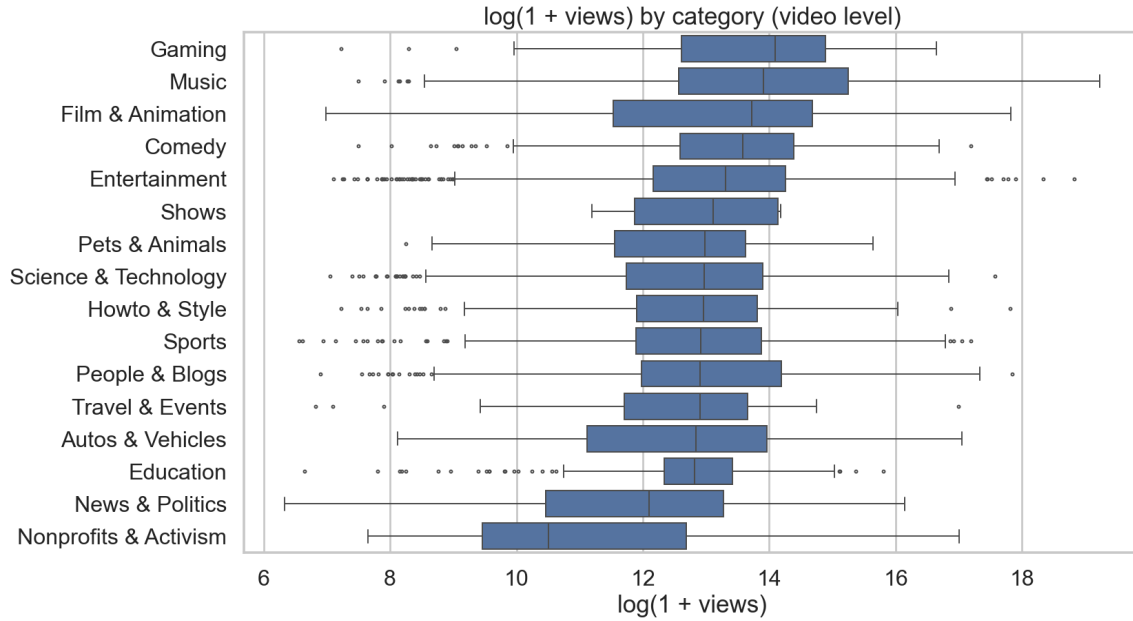


Figure 2: $\log(1 + \text{views})$ by category (video level), ordered by median.

4.3 Engagement ratios differ by category

Figure 3 plots like rate and comment rate by category. Like rates cluster around 2–4%, and the ordering of categories by like rate does not perfectly match the ordering by views — large-view categories are not uniformly the most “engaging” per view, which motivates treating ratios as distinct outcomes rather than proxies for views.

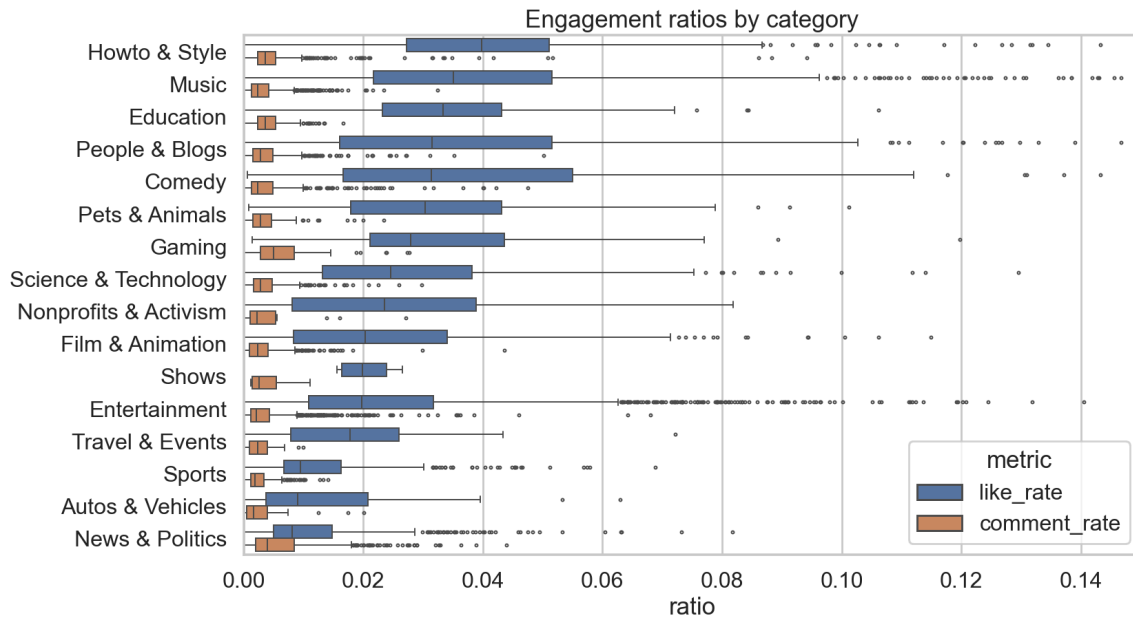


Figure 3: Like rate and comment rate by category (x-axis truncated at 0.15 for readability).

4.4 Comment/rating settings

Only 105 videos have comments disabled and 30 have ratings disabled at the video level. Figure 4 compares $\log(1 + \text{views})$ across these groups. The groups are small — a data-issue in its own right — but useful for a first look at whether disabled settings co-occur with weaker performance.

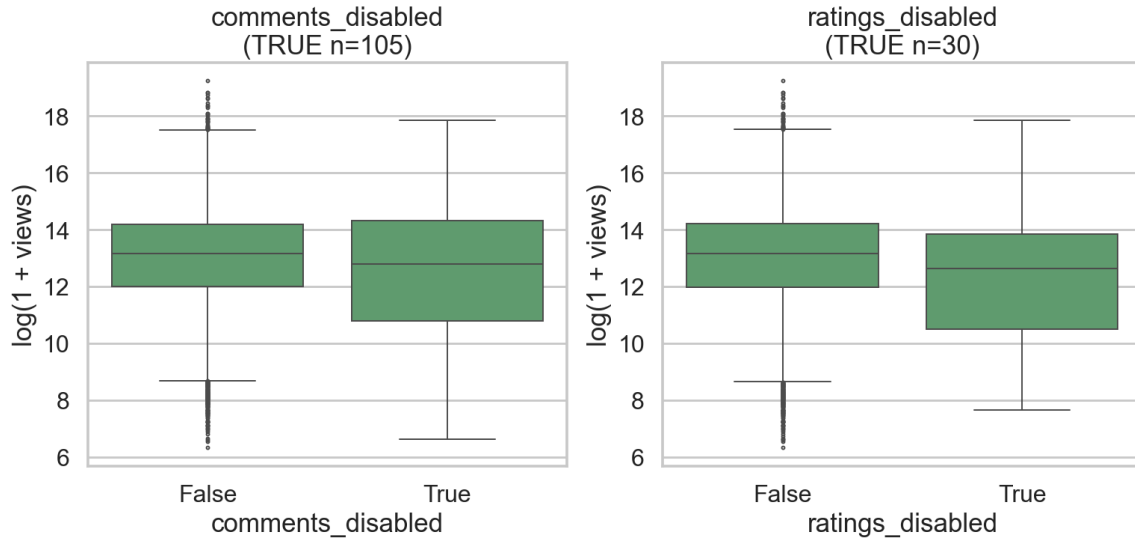


Figure 4: $\log(1 + \text{views})$ by `comments_disabled` and `ratings_disabled`.

4.5 Time effects are mild

Figure 5 shows mean $\log(1 + \text{views})$ by publish hour and day of week. The hour profile is nearly flat, with a modest bump in the early UTC morning, and Thursday/Friday publish days are slightly higher. Crucially, `publish_hour` is **UTC**, not US-local, so these patterns are confounded by timezone (Section 5). The effect sizes are small relative to the category spread.

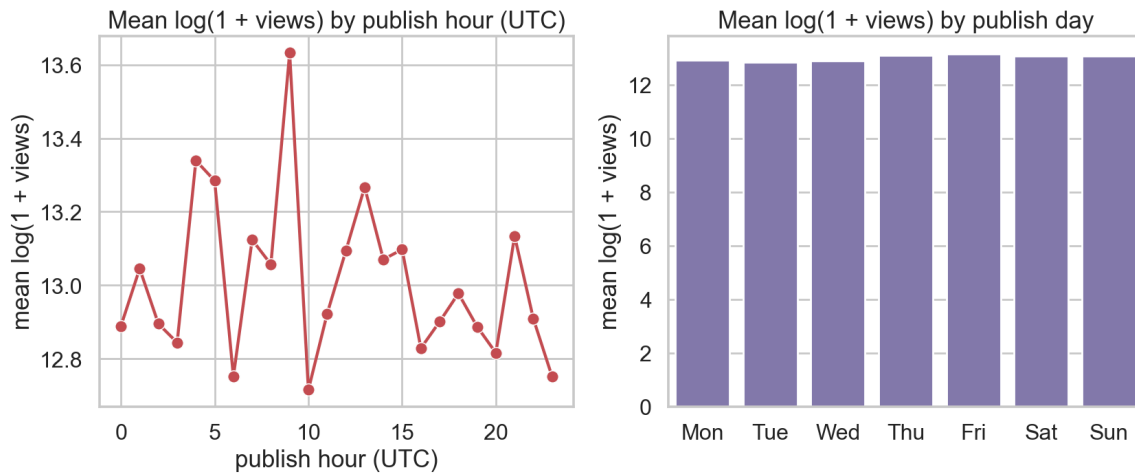


Figure 5: Mean $\log(1 + \text{views})$ by publish hour (UTC, left) and day of week (right).

4.6 Text and tag features carry weak linear signal

Figure 6 plots $\log(1 + \text{views})$ against title length and tag count. Title length has a weak negative association ($r = -0.06$) and tag count a weak positive one ($r = 0.14$). The relationships look non-linear

and heteroskedastic, which is exactly the kind of pattern that tree-based models may capture better than a linear fit.

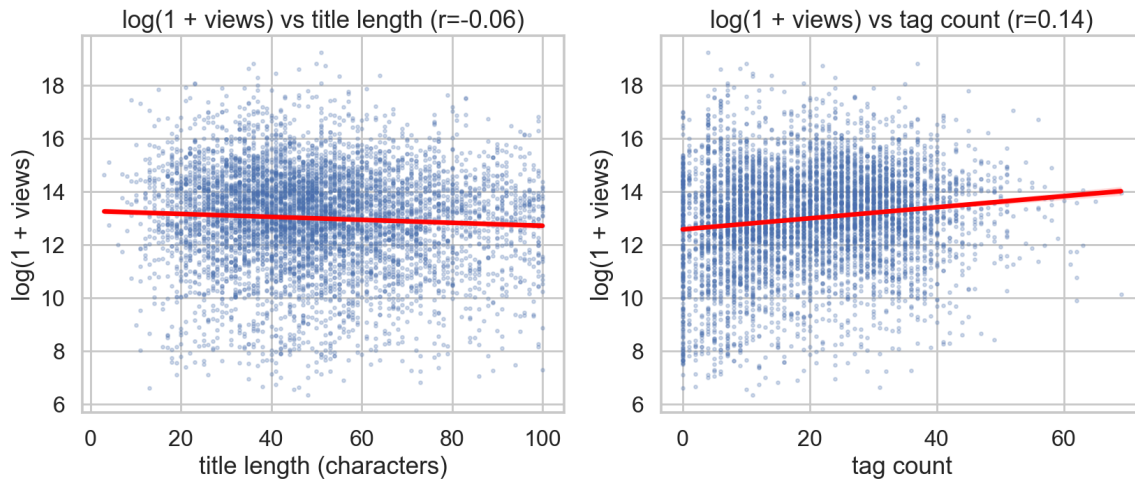


Figure 6: $\log(1 + \text{views})$ versus title length (left) and tag count (right), with linear trend lines.

4.7 Collinearity and leakage become visible in the correlation matrix

Figure 7 is the correlation matrix. It surfaces two important facts. First, the count outcomes are almost perfectly co-linear: $\log(\text{views})$ correlates **0.87** with $\log(\text{likes})$, **0.87** with $\log(\text{dislikes})$, and **0.79** with $\log(\text{comments})$. Second, days_to_trend is negatively correlated with views (-0.14), i.e. videos that trend quickly after publishing tend to have higher final views.

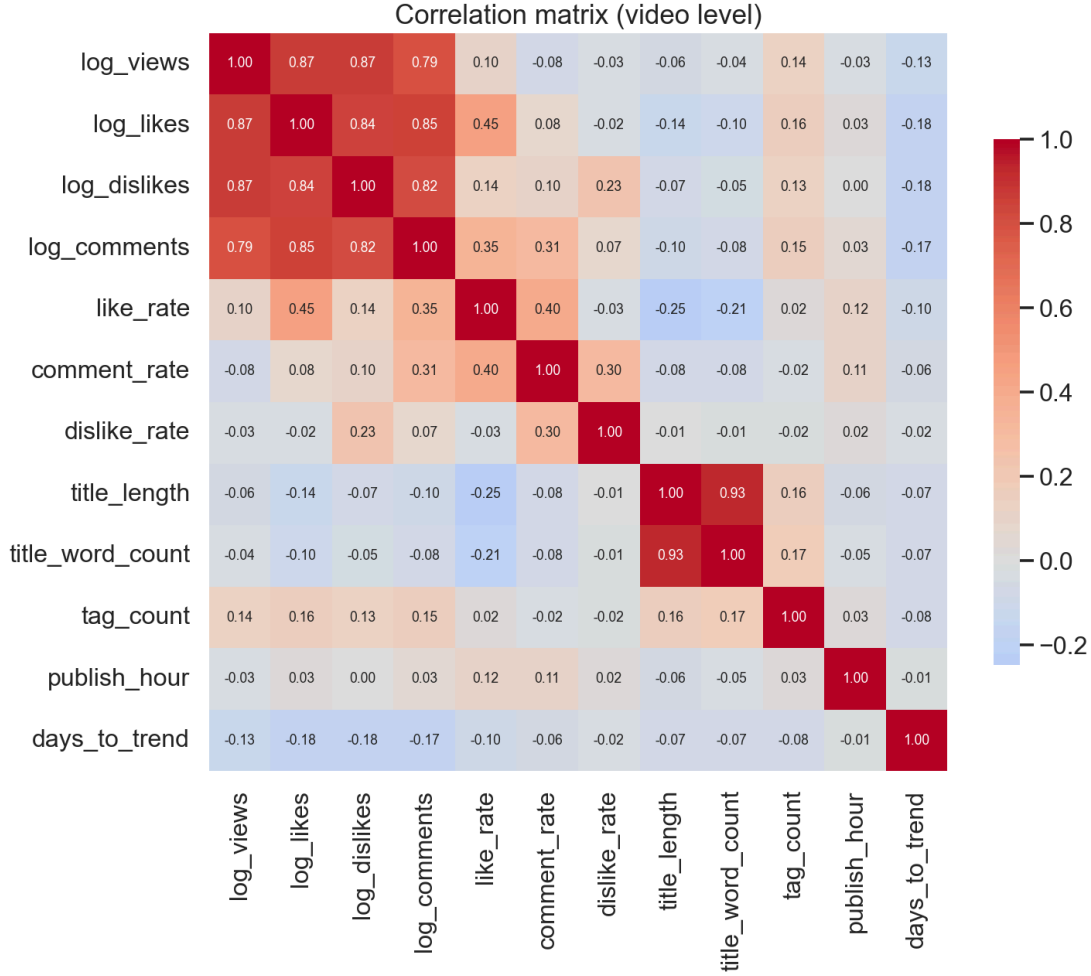


Figure 7: Correlation matrix of numeric features (video level). Strong outcome-outcome correlations confirm that counts cannot be used as one another’s predictors.

4.8 Class balance and snapshot multiplicity

Figure 8 shows the classification target is a clean 25%/75% split, and that snapshot multiplicity is highly variable across videos — the visual companion to the leakage discussion in Section 5.

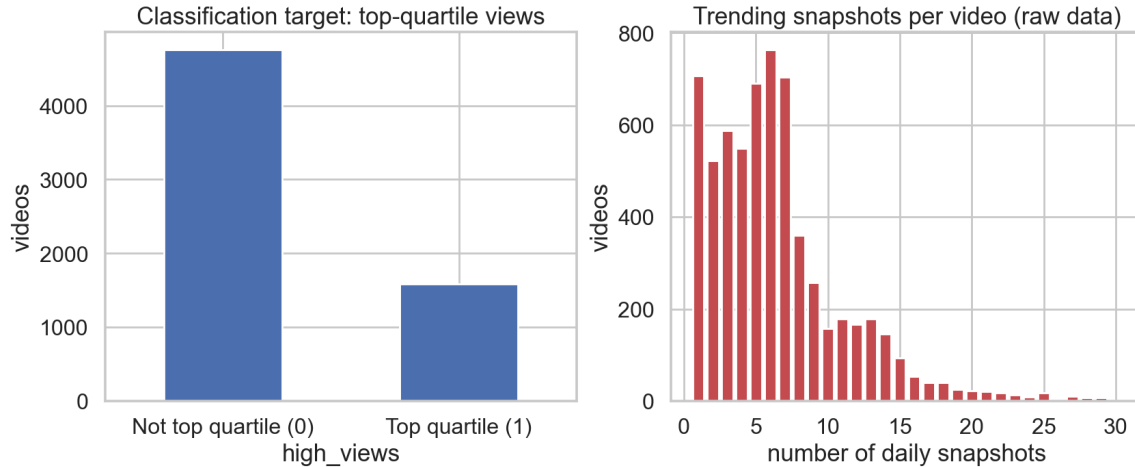


Figure 8: Left: the top-quartile-view target is balanced. Right: distribution of the number of daily snapshots per video in the raw data.

5 Data Issues, Limitations, and Leakage Risks

1. **Severe right-skew.** Counts span several orders of magnitude (skew up to 44.9 for dislikes). Mitigation: $\log(1 + x)$ responses, robust metrics, and tree models that are invariant to monotone transforms of the target.
2. **Temporal / repeated-measures leakage.** The raw table’s 40,949 rows describe only 6,351 videos, and counts are **cumulative**. Rows for the same video are dependent, so a naive random split would put the same video’s earlier and later snapshots in both train and test. Mitigation: model at the video level and use grouped cross-validation (Section 6).
3. **Contemporaneous leakage.** likes, dislikes, and comment_count are near-perfectly correlated with views (0.79–0.87) because they accumulate together. Using them to predict views (or using counts to predict ratios) is leakage, not signal. Mitigation: restrict X to upload-time metadata and justify the exclusion explicitly.
4. **Selection bias.** The file contains only videos that **trended**. There is no comparison set of non-trending videos, so all conclusions are conditional on trending, and we cannot estimate “what causes a video to trend.” The classification target is therefore “high engagement **among trending videos**.”
5. **Rare flags and class imbalance.** comments_disabled (1.5%) and ratings_disabled (0.4%) are near-constant; video_error_or_removed (0.06%) is a filter. Near-constant predictors can cause quasi-separation in logistic regression and unstable coefficients.
6. **Missing data and sentinels.** Effectively no missingness; the real issues are the [none] tag sentinel (1,535 rows) and the unused description nulls (570 rows).
7. **Outliers and anomalies.** Views reach 225 million; days_to_trend reaches **4,215 days** (about 11.5 years), with 69 videos published more than a year before trending — an “old catalog resurfacing” effect. These warrant a conscious keep/flag decision rather than silent deletion.
8. **Unobserved heterogeneity.** The data lack channel-level covariates such as subscriber count, and 2,207 channels recur, so channel popularity is folded into unexplained variance and repeated-channel rows can contaminate folds. Mitigation: group by channel.
9. **Timezone caveat.** publish_time is UTC, so publish_hour is not local US time; any “best time to publish” conclusion is provisional.

10. **Provenance.** The attribution of the dataset to a specific Kaggle contributor should be verified before submission (the shape and date range match the commonly cited “Trending YouTube Video Statistics” release).

6 Initial Modeling Plan

6.1 Analysis unit and targets

All models are fit on the **video-level** table ($n = 6,348$). Targets are:

1. Regression: $\log(1 + \text{views})$, like_rate , comment_rate (and $\log(1 + \text{dislikes})$ as a secondary response reflecting the proposal).
2. Classification: high_views (top-quartile final views, threshold 1,473,578).

Predictors are the upload-time metadata set from Table 1. We exclude all engagement counts, `thumbnail_link`, `description`, and identifiers, and we fit every encoder/imputer/scaler inside the training folds only.

6.2 Candidate methods and why

Regularized linear regression (Ridge / LASSO). The design has a 16-level categorical and several correlated/time-derived features. Ridge stabilizes coefficients under collinearity; LASSO performs feature selection and yields a sparse, interpretable set of metadata effects. We tune the penalty by nested cross-validation.

Tree ensembles (Random Forest, Gradient Boosting). EDA showed weak, non-linear, and heteroskedastic marginal relationships (title length, tag count) and probable interactions (for example, the effect of title length may differ by category). Trees capture these without manual specification and provide permutation-importance and partial-dependence summaries.

Classification models (penalized logistic regression and SVM). Logistic regression is interpretable and well-suited to a balanced binary target; SVMs with linear and RBF kernels provide a flexible non-linear benchmark. We tune the regularization strength (and gamma for RBF) by grouped CV.

For **inference** (RQ3), separate from the predictive comparison, we fit an ordinary least-squares / GLM model on the interpretable feature set with heteroskedasticity-robust standard errors and report standardized coefficients with 95% confidence intervals. We treat the regularized and tree models as predictive tools only, since shrinkage and non-linearity complicate formal significance testing.

6.3 Validation framework

1. **Grouped cross-validation.** Because snapshots repeat within videos and channels repeat across the data, we use `GroupKFold` grouped by `video_id` (snapshot level) and by `channel_title` (video level), with stratification on the binary target. This prevents the leakage identified in Section 5.
2. **Nested CV.** Hyperparameters are tuned in an inner loop so that the reported error is not optimistic.
3. **Temporal holdout (secondary).** As a robustness check we train on earlier trending dates and test on the most recent period, to see whether conclusions hold out of time.
4. **Reproducibility.** Fixed random seeds and a saved feature-engineering pipeline.

6.4 Evaluation metrics

Regression: RMSE and MAE on the log scale (and, after back-transformation, on the raw scale) plus R-squared. Classification: ROC-AUC, error rate, and F1 / confusion matrix at the selected threshold.

6.5 Planned comparisons

We will compare (i) a metadata-only baseline against (ii) the regularized and tree models, to quantify how much signal upload-time metadata actually contains, and we will report the inference model's coefficient table separately for the significance question. Residual diagnostics and partial-dependence plots will be used to check that any apparent non-linearity is real rather than an artifact of outliers.

7 Summary and Next Steps

This report imported and audited the US YouTube trending data (40,949 rows, 16 columns), identified the correct unit of analysis (6,348 videos after collapsing daily snapshots), preprocessed and engineered features with explicit leakage controls, and produced an EDA that establishes severe right-skew, category effects, weak time/title/tag signal, and strong outcome-collinearity. It also documented the principal data issues: repeated-measures and contemporaneous leakage, selection bias from observing only trending videos, rare flags, a tag sentinel, outliers, a UTC timezone caveat, and missing channel covariates.

Next steps (Progress Report 2). (1) Implement the grouped, nested cross-validation pipeline; (2) fit and tune the regression, tree, and classification models; (3) fit the inference model with robust standard errors and interpret the coefficients; (4) compare all models on the agreed metrics and visualize importances and partial dependence.

8 Appendix: Analysis Code

The following scripts reproduce every number and figure in this report. They are also in the project repository under analysis/.

8.1 profile_data.py — data audit

"""Profile the Kaggle YouTube Trending (US) dataset.

Run: python analysis/profile_data.py

All output goes to stdout so it can be captured into the report's data section.

"""

```
from __future__ import annotations
```

```
import json
```

```
from pathlib import Path
```

```
import numpy as np
```

```
import pandas as pd
```

```
pd.set_option("display.width", 160)
```

```
pd.set_option("display.max_columns", 50)
```

```
DATA = Path(__file__).resolve().parents[1] / "data"
```

```
CSV = DATA / "USvideos.csv"
```

```
CATS = DATA / "US_category_id.json"
```

```
def banner(title: str) -> None:
```

```
    print("\n" + "=" * 78)
```

```
    print(title)
```

```
    print("=" * 78)
```

```
def main() -> None:
```

```
    df = pd.read_csv(CSV)
```

```
    banner("1. DIMENSIONS & COLUMNS")
```

```
    print("shape (rows, cols):", df.shape)
```

```
    print("columns:", list(df.columns))
```

```
    banner("2. DTYPES")
```

```
    print(df.dtypes.to_string())
```

```
    banner("3. DATE RANGES (trending_date = YY.DD.MM)")
```

```
    td = pd.to_datetime(df["trending_date"], format="%y.%d.%m", errors="coerce")
```

```
    pt = pd.to_datetime(df["publish_time"], errors="coerce", utc=True)
```

```
    print("trending_date min/max:", td.min().date(), td.max().date(), "| unparsed:", int(td.isna().sum()))
```

```
    print("publish_time min/max:", pt.min(), pt.max(), "| unparsed:", int(pt.isna().sum()))
```

```
    print("distinct trending dates:", int(td.nunique()))
```

```
    print("all trending_date match ^\\d{2}\\d{2}\\d{2}$:",
```

```
          bool(df["trending_date"].astype(str).str.match(r"^\d{2}\.\d{2}\.\d{2}$").all()))
```

```
    banner("4. MISSING / SENTINELS")
```

```
    print("NaN per column:\n", df.isna().sum().to_string())
```

```
    tags = df["tags"].astype(str).str.strip()
```

```
    print("\ntags == '[none]':", int((tags == "[none]").sum()))
```

```
    print("tags == '[None]':", int((tags == "[None]").sum()))
```

```
    print("tags empty string:", int((tags == "").sum()))
```

```
    obj = df.select_dtypes(include="object").columns
```

```
    empt = df[obj].apply(lambda s: s.astype(str).str.strip() == "").sum()
```

```
    print("\nempty-string counts (object cols):\n", empt.to_string())
```

```
    banner("5. DUPLICATES & REPEATED SNAPSHOTS")
```

```
    print("fully duplicated rows:", int(df.duplicated().sum()))
```

```
    print("duplicated video_id :", int(df["video_id"].duplicated().sum()))
```

```
    print("unique video_id      :", int(df["video_id"].nunique()))
```

```
    vc = df["video_id"].value_counts()
```

```
    print("max appearances of one video_id:", int(vc.max()))
```

```
    print("\ndistribution of #appearances:\n", vc.value_counts().sort_index().to_string())
```

```
    print("\ntop 5 most-trending video_ids:\n", vc.head().to_string())
```

```
    # duplicate (video_id, trending_date) pairs -> true duplicate snapshots
```

```
    dup_pairs = df.duplicated(subset=["video_id", "trending_date"]).sum()
```

```
    print("\nduplicated (video_id, trending_date) pairs:", int(dup_pairs))
```

```
    banner("6. NUMERIC SUMMARY + SKEW")
```

```
    num = ["views", "likes", "dislikes", "comment_count"]
```

```
    desc = df[num].describe(percentiles=[0.25, 0.5, 0.75, 0.9, 0.99]).T
```

```
    print(desc.to_string())
```

```

print("\nskewness:\n", df[num].skew().to_string())
print("\nkurtosis:\n", df[num].kurt().to_string())

banner("7. BOOLEAN FLAGS")
for c in ["comments_disabled", "ratings_disabled", "video_error_or_removed"]:
    print(f"\n{c}:")
    print(df[c].value_counts(dropna=False).to_string())
    print((df[c].value_counts(normalize=True, dropna=False) * 100).round(3).to_string(), "%")

banner("8. CATEGORY DISTRIBUTION (id -> name)")
with open(CATS, encoding="utf-8") as fh:
    cats = json.load(fh)
id2name = {int(i["id"]): i["snippet"]["title"] for i in cats["items"]}
print("category_id value_counts:\n", df["category_id"].value_counts().sort_index().to_string())
print("\nnum distinct categories in data:", int(df["category_id"].nunique()))
print("\ncategories appearing in data (name):")
print(df["category_id"].map(id2name).value_counts().to_string())

banner("9. LOGICAL INCONSISTENCIES")
print("views == 0", int((df["views"] == 0).sum()))
print("likes == 0", int((df["likes"] == 0).sum()))
print("comment_count == 0", int((df["comment_count"] == 0).sum()))
print("likes > views", int((df["likes"] > df["views"]).sum()))
print("dislikes > views", int((df["dislikes"] > df["views"]).sum()))
print("comment_count > views", int((df["comment_count"] > df["views"]).sum()))
print("likes+dislikes > views:", int(((df["likes"] + df["dislikes"]) > df["views"]).sum()))
# comments disabled but comments recorded
cd_true = df["comments_disabled"].astype(str).str.lower() == "true"
print("comments_disabled=True AND comment_count>0:",
      int((cd_true & (df["comment_count"] > 0)).sum()))

banner("10. CHANNELS & ENGAGEMENT RATIOS")
print("distinct channel_title:", int(df["channel_title"].nunique()))
print("top 10 channels by #trending rows:\n", df["channel_title"].value_counts().head(10).to_string())
eng = pd.DataFrame({
    "like_rate": df["likes"] / df["views"].replace(0, np.nan),
    "comment_rate": df["comment_count"] / df["views"].replace(0, np.nan),
})
print("\nengagement-ratio summary:")
print(eng.describe(percentiles=[0.25, 0.5, 0.75, 0.9]).T.to_string())
print("\nlike_rate > 1 (should be rare/impossible):", int((eng["like_rate"] > 1).sum()))
print("comment_rate > 1:", int((eng["comment_rate"] > 1).sum()))

banner("11. PER-VIDEO DEDUP PREVIEW (last snapshot per video)")
last_idx = df.groupby("video_id")["trending_date"].transform("max") == df["trending_date"]
dedup = df[last_idx]
print("rows after keeping last snapshot per video_id:", len(dedup))
print("unique videos:", int(dedup["video_id"].nunique()))
print("views summary (dedup):\n", dedup["views"].describe(percentiles=[.25, .5, .75, .9, .99]).to_string())

if __name__ == "__main__":
    main()

```

8.2 clean_eda.py — cleaning, feature engineering, figures

"""Phase 1-2: cleaning, feature engineering, and EDA for the US YouTube trending data.

Produces:

analysis/output/videos.csv	video-level (deduplicated) analysis table
analysis/output/summary_*.csv	numeric summaries used in the report
report/figures/fig*.png	report figures

Run: python analysis/clean_eda.py

"""

from __future__ import annotations

import json
from pathlib import Path

import matplotlib

matplotlib.use("Agg")
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import seaborn as sns

ROOT = Path(__file__).resolve().parents[1]
DATA = ROOT / "data"
OUT = ROOT / "analysis" / "output"

```

FIGS = ROOT / "report" / "figures"
OUT.mkdir(parents=True, exist_ok=True)
FIGS.mkdir(parents=True, exist_ok=True)

sns.set_theme(style="whitegrid", context="talk")
plt.rcParams.update({"figure.dpi": 140, "savefig.bbox": "tight", "font.size": 12})

LOG = "\n[clean_eda] "
step_log: list[str] = []

def log(msg: str) -> None:
    print(LOG + msg)
    step_log.append(msg)

def savefig(fig, name: str) -> None:
    path = FIGS / f"{name}.png"
    fig.savefig(path)
    plt.close(fig)
    log(f"wrote figure {path.relative_to(ROOT)}")

def main() -> None:
    raw = pd.read_csv(DATA / "USvideos.csv")
    n0 = len(raw)
    log(f"raw rows = {n0}, columns = {raw.shape[1]}")

    # ---- parse dates ----
    # trending_date uses an unusual YY.DD.MM format (17.14.11 -> 2017-11-14).
    raw["trending_date_parsed"] = pd.to_datetime(raw["trending_date"], format="%y.%d.%m")
    raw["publish_time_parsed"] = pd.to_datetime(raw["publish_time"], utc=True)
    log(f"trending range {raw.trending_date_parsed.min().date()} .. {raw.trending_date_parsed.max().date()}")
    log(f"publish range {raw.publish_time_parsed.min()} .. {raw.publish_time_parsed.max()}")

    # ---- cleaning steps with counts ----
    clean = raw.copy()
    counts = {"raw": n0}

    n_removed_flag = int((clean["video_error_or_removed"]).sum())
    clean = clean[~clean["video_error_or_removed"].copy()]
    counts["drop_error_or_removed"] = len(clean)
    log(f"drop video_error_or_removed: -{n0 - len(clean)} rows (flagged={n_removed_flag})")

    before = len(clean)
    clean = clean.drop_duplicates()
    counts["drop_full_dup"] = len(clean)
    log(f"drop fully duplicated rows: -{before - len(clean)} rows")

    before = len(clean)
    clean = clean.drop_duplicates(subset=["video_id", "trending_date_parsed"], keep="last")
    counts["drop_dup_snapshot"] = len(clean)
    log(f"drop duplicate (video_id, trending_date) snapshots: -{before - len(clean)} rows")

    # ---- feature engineering ----
    clean["publish_hour"] = clean["publish_time_parsed"].dt.hour
    clean["publish_dayofweek"] = clean["publish_time_parsed"].dt.dayofweek # 0=Mon
    clean["publish_date"] = clean["publish_time_parsed"].dt.tz_localize(None).dt.normalize()
    clean["days_to_trend"] = (clean["trending_date_parsed"] - clean["publish_date"]).dt.days

    clean["title_length"] = clean["title"].str.len()
    clean["title_word_count"] = clean["title"].str.split().str.len()
    # tags is a '|' -separated list; '[none]' means no tags.
    tags_clean = clean["tags"].fillna("").str.strip()
    clean["tag_count"] = np.where(tags_clean == "[none]", 0, tags_clean.str.count(r"\|") + 1)
    clean.loc[tags_clean == "", "tag_count"] = 0

    with open(DATA / "US_category_id.json", encoding="utf-8") as fh:
        cats = json.load(fh)
    id2name = {int(i["id"]): i["snippet"]["title"] for i in cats["items"]}
    clean["category"] = clean["category_id"].map(id2name)

    clean["log_views"] = np.log1p(clean["views"])
    clean["log_likes"] = np.log1p(clean["likes"])
    clean["log_dislikes"] = np.log1p(clean["dislikes"])
    clean["log_comments"] = np.log1p(clean["comment_count"])
    clean["like_rate"] = clean["likes"] / clean["views"]
    clean["comment_rate"] = clean["comment_count"] / clean["views"]
    clean["dislike_rate"] = clean["dislikes"] / clean["views"]

    log(f"snapshot-level table after cleaning: {len(clean)} rows")

```

```

# ---- deduplicate to one row per video (final cumulative snapshot) ----
idx_last = clean.sort_values("trending_date_parsed").groupby("video_id").tail(1).index
videos = clean.loc[idx_last].copy()
counts["video_level"] = len(videos)
log(f"video-level table (last trending snapshot per video): {len(videos)} videos")

# classification target: top-quartile final views
q75 = videos["views"].quantile(0.75)
videos["high_views"] = (videos["views"] >= q75).astype(int)
log(f"top-quartile views threshold = {q75:.0f}; positives = {videos.high_views.mean():.1%}")

videos.to_csv(OUT / "videos.csv", index=False)

# persistence/summary tables -----
pd.DataFrame(
    [{"step": k, "rows": v} for k, v in counts.items()]
).to_csv(OUT / "summary_cleaning_counts.csv", index=False)

num = ["views", "likes", "dislikes", "comment_count"]
videos[num].describe(percentiles=[.25, .5, .75, .9, .99]).T.join(
    videos[num].skew().rename("skew")
).to_csv(OUT / "summary_numeric.csv")

videos.groupby("category")["log_views"].agg(["count", "mean", "median", "std"]).sort_values(
    "median", ascending=False
).to_csv(OUT / "summary_by_category.csv")

videos.groupby("publish_hour")["log_views"].mean().to_csv(OUT / "summary_by_hour.csv")
videos.groupby("publish_dayofweek")["log_views"].mean().to_csv(OUT / "summary_by_dow.csv")

# ---- figures -----
# Fig 1: raw vs log views
fig, axes = plt.subplots(1, 2, figsize=(13, 5))
sns.histplot(videos["views"], bins=60, ax=axes[0], color="#c44e52")
axes[0].set_title(f"Raw views (skew = {videos.views.skew():.1f})")
axes[0].set_xlabel("views")
sns.histplot(videos["log_views"], bins=60, ax=axes[1], color="#4c72b0")
axes[1].set_title(f"log(1 + views) (skew = {videos.log_views.skew():.2f})")
axes[1].set_xlabel("log(1 + views)")
savefig(fig, "fig01_views_transform")

# Fig 2: log views by category
order = videos.groupby("category")["log_views"].median().sort_values(ascending=False).index
fig, ax = plt.subplots(figsize=(12, 7))
sns.boxplot(data=videos, y="category", x="log_views", order=order, ax=ax,
            color="#4c72b0", fliersize=2)
ax.set_title("log(1 + views) by category (video level)")
ax.set_xlabel("log(1 + views)")
ax.set_ylabel("")
savefig(fig, "fig02_logviews_by_category")

# Fig 3: engagement rates by category
eng = videos.melt(id_vars="category", value_vars=["like_rate", "comment_rate"],
                  var_name="metric", value_name="rate")
order2 = videos.groupby("category")["like_rate"].median().sort_values(ascending=False).index
fig, ax = plt.subplots(figsize=(12, 7))
sns.boxplot(data=eng, y="category", x="rate", hue="metric", order=order2, ax=ax, fliersize=2)
ax.set_xlim(0, 0.15)
ax.set_title("Engagement ratios by category")
ax.set_xlabel("ratio")
ax.set_ylabel("")
savefig(fig, "fig03_engagement_by_category")

# Fig 4: flags vs log views
flags = videos.melt(id_vars=["log_views"],
                    value_vars=["comments_disabled", "ratings_disabled"],
                    var_name="flag", value_name="value")
fig, axes = plt.subplots(1, 2, figsize=(13, 5))
for ax, flag in zip(axes, ["comments_disabled", "ratings_disabled"]):
    sub = videos[videos[flag]]
    sns.boxplot(data=videos, x=flag, y="log_views", ax=ax, color="#55a868",
                fliersize=2)
    ax.set_title(f"{flag}\n(TRUE n={len(sub)})")
    ax.set_xlabel(flag)
    ax.set_ylabel("log(1 + views)")
savefig(fig, "fig04_flags_vs_views")

# Fig 5: publish hour & day of week
fig, axes = plt.subplots(1, 2, figsize=(14, 5))
hourly = videos.groupby("publish_hour")["log_views"].mean()
sns.lineplot(x=hourly.index, y=hourly.values, marker="o", ax=axes[0], color="#c44e52")
axes[0].set_title("Mean log(1 + views) by publish hour (UTC)")

```

```

axes[0].set_xlabel("publish hour (UTC)")
axes[0].set_ylabel("mean log(1 + views)")
dow = videos.groupby("publish_dayofweek")["log_views"].mean()
names = ["Mon", "Tue", "Wed", "Thu", "Fri", "Sat", "Sun"]
sns.barplot(x=names, y=dow.values, ax=axes[1], color="#8172b3")
axes[1].set_title("Mean log(1 + views) by publish day")
axes[1].set_xlabel("")
axes[1].set_ylabel("mean log(1 + views)")
savefig(fig, "fig05_time_effects")

# Fig 6: title length & tag count
fig, axes = plt.subplots(1, 2, figsize=(14, 5))
sns.regplot(data=videos, x="title_length", y="log_views", ax=axes[0],
            scatter_kws={"s": 6, "alpha": 0.25}, line_kws={"color": "red"})
axes[0].set_title(f"log(1 + views) vs title length (r={videos.title_length.corr(videos.log_views):.2f})")
axes[0].set_xlabel("title length (characters)")
axes[0].set_ylabel("log(1 + views)")
sns.regplot(data=videos, x="tag_count", y="log_views", ax=axes[1],
            scatter_kws={"s": 6, "alpha": 0.25}, line_kws={"color": "red"})
axes[1].set_title(f"log(1 + views) vs tag count (r={videos.tag_count.corr(videos.log_views):.2f})")
axes[1].set_xlabel("tag count")
axes[1].set_ylabel("log(1 + views)")
savefig(fig, "fig06_text_metrics")

# Fig 7: correlation heatmap
corr_cols = ["log_views", "log_likes", "log_dislikes", "log_comments",
            "like_rate", "comment_rate", "dislike_rate", "title_length",
            "title_word_count", "tag_count", "publish_hour", "days_to_trend"]
fig, ax = plt.subplots(figsize=(11, 9))
sns.heatmap(videos[corr_cols].corr(), annot=True, fmt=".2f", cmap="coolwarm",
            center=0, square=True, ax=ax, cbar_kws={"shrink": 0.7}, annot_kws={"size": 9})
ax.set_title("Correlation matrix (video level)")
savefig(fig, "fig07_correlation")

# Fig 8: class balance + snapshot multiplicity
fig, axes = plt.subplots(1, 2, figsize=(14, 5))
videos["high_views"].value_counts().sort_index().plot.bar(ax=axes[0], color="#4c72b0")
axes[0].set_title("Classification target: top-quartile views")
axes[0].set_xticklabels(["Not top quartile (0)", "Top quartile (1)", rotation=0])
axes[0].set_ylabel("videos")
mult = raw.groupby("video_id").size().value_counts().sort_index()
axes[1].bar(mult.index, mult.values, color="#c44e52")
axes[1].set_title("Trending snapshots per video (raw data)")
axes[1].set_xlabel("number of daily snapshots")
axes[1].set_ylabel("videos")
savefig(fig, "fig08_class_and_multiplicity")

# write a human-readable cleaning log
(OUT / "cleaning_log.txt").write_text("\n".join(step_log), encoding="utf-8")
log("done")

if __name__ == "__main__":
    main()

```

8.3 report_numbers.py — summary statistics cited above

```

"""Print the compact set of numbers cited in Progress Report 1."""
from __future__ import annotations

from pathlib import Path

import pandas as pd

ROOT = Path(__file__).resolve().parents[1]
videos = pd.read_csv(ROOT / "analysis" / "output" / "videos.csv")
raw = pd.read_csv(ROOT / "data" / "USvideos.csv")

pd.set_option("display.width", 160)

print("### video level: n =", len(videos))
print("\n-- engagement ratios --")
print(videos[["like_rate", "comment_rate", "dislike_rate"]].describe(
    percentiles=[.25, .5, .75]).T.to_string())

print("\n-- correlations with log_views (video level) --")
cols = ["title_length", "title_word_count", "tag_count", "publish_hour",
        "days_to_trend", "log_likes", "log_dislikes", "log_comments"]
print(videos[cols].corrwith(videos["log_views"]).round(3).to_string())

print("\n-- title / tag features --")

```

```

print(videos[["title_length", "title_word_count", "tag_count"]].describe(
    percentiles=[.25, .5, .75]).T.to_string())
print("tag_count == 0:", int((videos.tag_count == 0).sum()))
print("title_length == 0:", int((videos.title_length == 0).sum()))

print("\n-- days_to_trend --")
print(videos["days_to_trend"].describe().to_string())
print("negative days_to_trend:", int((videos.days_to_trend < 0).sum()))
print("days_to_trend > 365:", int((videos.days_to_trend > 365).sum()))

print("\n-- flags at video level --")
for c in ["comments_disabled", "ratings_disabled"]:
    print(c, "n_true =", int(videos[c].sum()))

print("\n-- zeros at video level --")
for c in ["likes", "dislikes", "comment_count"]:
    print(f"{c} == 0:", int((videos[c] == 0).sum()))

print("\n-- top categories (video level) --")
print(videos["category"].value_counts().head(8).to_string())

print("\n-- raw snapshots --")
print("unique videos in raw:", raw.video_id.nunique())
vc = raw.groupby("video_id").size()
print("mean snapshots/video: %.2f" % vc.mean(), "| max:", int(vc.max()))
print("videos trending >= 2 days: %d (%.1f%%)" % ((vc >= 2).sum(), 100 * (vc >= 2).mean()))

print("\n-- raw views percentiles (snapshot) --")
print(raw["views"].describe(percentiles=[.25, .5, .75, .99]).to_string())

```

9 References

1. Jolly, M. (and/or the corresponding Kaggle release). **Trending YouTube Video Statistics**. Dataset, Kaggle. (Verify the exact contributor/citation before submission.)
2. YouTube. **Trending** and platform documentation, 2018.